Cross-displinary applications of the Toulbar2 solver

UE : Introductory Research Project

Authors :

Hugo Garcia
hugo.garcia@univ-tlse3.fr

Philippe Gauthier
philippe.gauthier@univ-tlse3.fr

Marco Sanfilippo
marco.sanfilippo@univ-tlse3.fr

Mathis Francine-Habas
mathis.francine-habas@univ-tlse3.fr

Supervisor :

Martin Cooper
(Lecturer and researcher at the Toulouse Institute of Computer Science Research - IRIT)
martin.cooper@univ-tlse3.fr

University : University of Toulouse
Adress : 118 Narbonne road, 31062 Toulouse, France

Date : March 30, 2026

Abstract

resumé à mettre ici

Key Words : *Mathematics, Informatics, ..*

Contents

List of Figures

List of Tables

List of Acronyms

WCSP Weighted Constraint Satisfaction Problem

CSP Constraint Satisfaction Problem

VCSP Valued Constraint Satisfaction Problem

CFN Cost Function Network

EPT Equivalence Preserving Transformations

SAC Soft Arc Consistency

FDAC Full Directional Arc Consistency

EDAC Existential Directional Arc Consistency

VAC Virtual Arc Consistency

OSAC Optimal Soft Arc Consistency

MILP Mixed-Integer Linear Programming

CP Constraint Programming

LP Linear Programming

MIP Mixed Integer Programming

MINLP Mixed Integer Non-Linear Programming

SBF Smart Brute-force

ZP Zahn's problem

Introduction

The constraint satisfaction problem (CSP) involves finding an assignment for a set of variables over a finite domain that respects a set of given hard constraints. To find a solution to a CSP problem, backtracking search is used. This strategy will create a new subtree every time a value is assigned to a variable. Once the current instance is infeasible, the algorithm will backtrack and continue with a different assignment until a solution to the original problem is found. However, many real-world problems require optimization rather than mere satisfaction.

The valued constraint satisfaction problem (VCSP) or weighted constraint satisfaction problem (WCSP) is considered a generalization of the classic CSP. In a VCSP, a cost function network (CFN) framework is used. A CFN is a tuplet consisting of a finite set of discrete variables X, a finite domain D, a finite set of soft constraint C, and a maximum cost k. Soft constraints assign a cost to a specific assignment of a subset of variables. The NP-Hard problem of the VCSP is to find the instantiation of minimal cost.

To solve a VCSP, the branch-and-bound algorithm (BnB) is used. BnB explores the solution space by 'branching': it assigns a value to a variable and creates a new subtree. At any node, if the current instance has cost higher than maximum cost k or its cost is higher than the cost of the best solution found so far, the branch is pruned and the search backtracks. This process continues until all branches have been pruned, and the best global solution is found.

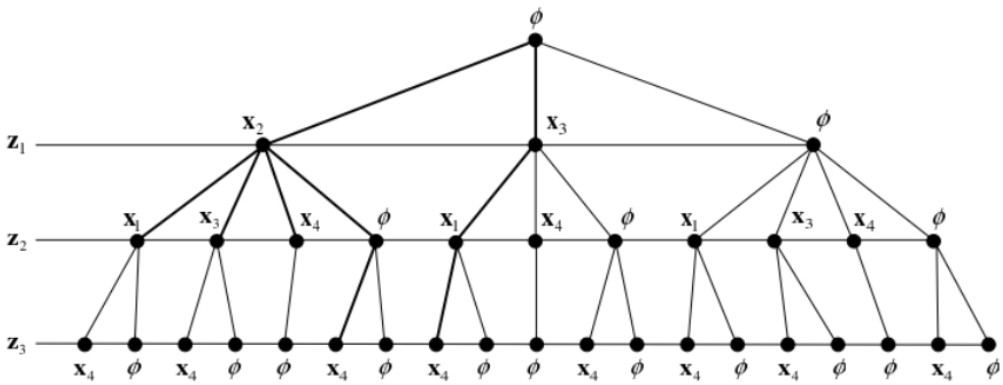

Figure 1: Example of BnB: Mobile Robot Localisation and Mapping in Extensive Outdoor Environments [**Bailey**, **T. (2002)**].

The efficacy of the branch-and-bound depends on the lower bound we have access to. In a Cost Function Network, functions are categorized by their arity: unary functions (arity 1) assign costs to individual variable assignments, binary functions (arity 2) assign costs to pairs of variables, and the nullary constant (arity 0), denoted as $c_\emptyset$, represents a global lower bound on the cost of any complete assignment.

Equivalence preserving transformations (EPT) transfer costs from one constraint to another while preserving the same optimal solution for the problem. Below, three different EPTs are presented for an example with three variables (x, y, z) with two possible assignments (a, b). A line joining two values represents a cost of 1 for this assignment in the corresponding binary cost function. Unary costs are represented as numbers inside the different assignments.

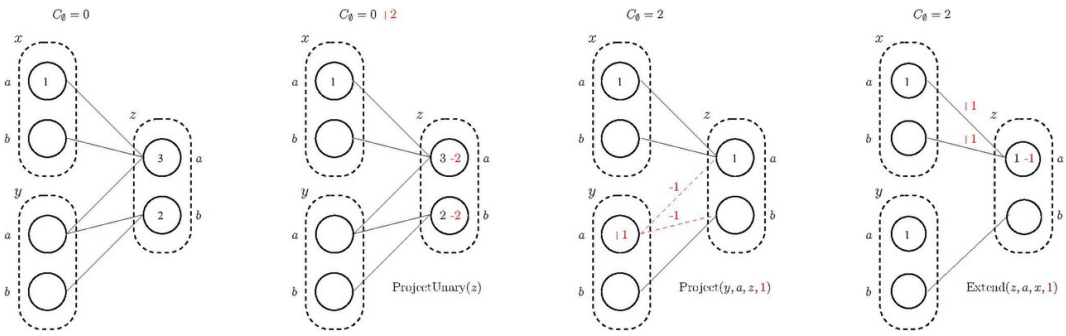

Figure 2: Basic Equivalence Preserving Transformations[**METTRE WIKIPEDIA**].

By successive EPT, the cost can be shifted and concentrated into the nullary constraint c, thus giving a better lower bound for the VCSP and allowing to quickly prune subtrees.

This process of applying successive EPTs to obtain a better lower bound is known as soft arc consistency (SAC). Different inference techniques to obtain a good lower bound exist, and those include the following: full directional arc consistency (FDAC), existential directional arc consistency (EDAC), virtual arc consistency (VAC) and optimal soft arc consistency (OSAC). Those inference techniques vary in relative strength vs relative computational cost, but generally the better the lower bound found, the higher the computational cost becomes. OSAC provides the best lower bound, followed by VAC, then EDAC and finally FDAC.

These theoretical findings are used in the solver Toulbar2. Toulbar2 is an open-source C++ solver for discrete optimization problems within the cost function network framework. The solver integrates SAC operations, dynamically switching and choosing the best inference technique at a given node, and uses a branch-and-bound search to quickly explore the solution space and find the optimum. This document provides a state-of-the-art review of the solver's cross-disciplinary impact. We begin by examining the solver's ability to solve difficult combinatorial computer science problems against other state-of-the-art solvers, then explore their real world applications ranging from structural bioinformatics to classical operations research and finally discuss their recent integration into neuro-symbolic Artificial Intelligence networks.

1 Algorithmic Foundations & Solver Development

1.1 An application to graph theory and the cluster editing problem

Zahn's decision problem (ZP) can be described as the cluster editing problem. Given a graph, what is the minimal number of operations (adding edges, removing edges) needed to create a disjoint union of cliques. The ZP is generally an NP-hard problem. In a paper titled 'A constraint programming approach to the Zahn's decision problem' , the ZP was transformed into a valued constraint satisfaction problem. This transformation is done as follows :

- A boolean variable is added for each pair of vertices. When a variable is set to true by the solver, then the decision is to keep or add the edge between the vertices. Else, the decision is to remove or not add an edge between the two vertices.
- A unary soft constraint is added for each variable. The cost of assigning 0 to a variable x is 1 if the edge exists in the original graph, and 0 otherwise.
- A ternary hard constraint is added for each pair of 3 edges which together creates the subgraph P3 described in the figure below. These ternary hard constraints are added to ensure that the resulting graph is a disjoint union of cliques. This result comes from the fact that a disjoint union of cliques cannot contain a subgraph isomorphic to P3, as explained in the paper.

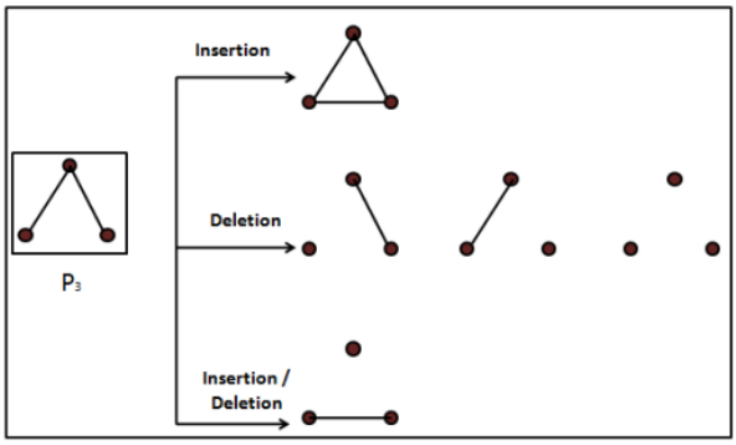

Figure 3: P3 Graph and insertion/deletion possibilities.

Using Toulbar2 to solve the VCSP, the results found were much better than the fixed parameter algorithm for cluster editing presented before. The VCSP algorithm performed better on both sparse and dense graphs. These results show the efficacy and utility of the constraint programming paradigm for general graph related problems.

1.2 Constraint programming for discrete integration in high dimensions

Discrete integration in high dimensions is considered to be a generally unsolved and difficult problem and has applications to a wide variety of fields. This difficulty arises from the phenomenon known as the curse of dimensionality. In the paper Taming the Curse of Dimensionality : Discrete Integration by Hashing and Optimization (Ermon, Gomez, Sabharwal, Selman), transform the original problem in set of combinatorial problems solved by Toulbar2

The problem at hand is described as follows: calculate the total sum over a large set of elements of a partition function w(). This problem is considered to be intractable, as the

sum of elements over an exponential number of different configurations. By changing the P complete counting problem into a series of NP-Hard MAP queries (Maximum a posteriori) optimization queries, the WISH algorithm bypasses the need for exhaustive summations.

Instead of attempting to enumerate an exponential number of configurations, the authors use randomized parity constraints linear equations over the field F2 to act as a hash function that "thins out" the configuration space. By solving a small, polynomial number of these constrained optimization instances, the solver Toulbar2 identifies the heaviest surviving configuration (the mode) at different levels of hashing. These specific maximum weights act as statistical proxies for the quantiles of the original weight distribution, allowing the total partition function to be reconstructed as a weighted sum of these median values with high-probability approximation guarantees (À VOIR)

Using Toulbar2, a state-of-the-art combinatorial optimization tool, WISH provided a constant-factor approximation of discrete integrals with high probability, that could since then only be handled heuristically.

1.3 Relevance of Toulbar2 today for domain consistency constraint programming and quadratic assignment problem

1.4 Toulbar2 application for the knapsack problem with a conflict graph and computational protein design

The knapsack problem is a classic NP-hard problem in combinatorics and computer science goes as follows: given a maximum weight capacity C, a set of objects each an assigned weight and value, what is the set of objects that maximize the value while respecting the weight capacity. The knapsack problem with a conflict graph (KPCG) is a variant of the knapsack problem where conflict between objects is added. For a given graph G, each object is assigned to a vertex and an edge between vertex i and j implies that object i and j cannot be in a solution together. The KPCG is difficult to solve with constraint programming as CFN do not allow the modeling of linear constraint without a non-trivial performance penalty, especially those with large coefficients. So and so developed a way to model pseudo boolean linear constraints with CNF allowing more problems to be solved using VSCP. When toulbar2 was compared with the state-of-the-art ILP solver CPLEX 20.1 for solving the knapsack problem with conflict graph. 6 classes with 720 instances each of the problem were tested with different weights, capacities and cost. The results found that Toulbar2 was more efficient than CPLEX in 4 out 6 classes. Furthermore, Toulbar2 found the best solutions for the largest number of instances in every class.

2 Resource Management & Planning

2.1 Preamble

Although the Toulbar2 solver has gained a strong reputation in the field of bioinformatics, the Cost Function Network formalism is proving particularly well-suited to classical operational research problems. The management and allocation of resources generally involve satisfying strict rules (hard constraints) whilst optimising quality criteria or preferences (soft constraints). This section illustrates how soft local consistency algorithms (such **OSAC!** (**OSAC!**) & **EDAC!** (**EDAC!**)) implemented in Toulbar2 enable the resolution of industrial-scale planning problems, whilst highlighting the strengths and limitations of these exact approaches compared to other paradigms such as **MILP!** (**MILP!**).

2.2 Spatio-temporal allocation and conflict management

Planning and allocating physical resources often require merging two complex dimensions : space and time. This could mean rotating crops across agricultural land over several years.

It could also involve coordinating flight paths in restricted airspace. These problems require satisfying strict rules while simultaneously optimizing quality criteria.
This subsection illustrates how the **WCSP!** (**WCSP!**) formalism models these spatio-temporal environments. It also evaluates the efficiency of the Toulbar2 solver compared to other exact methods.

The crop allocation problem consists of assigning different crop types (the values) to agricultural plots (the variables) over a finite time horizon. The approach proposed by Akplogan et al. (2013)[1] stands out by simultaneously modeling spatial and temporal constraints at the farm level.

The model integrates two types of constraints:

- **Hard constraints :** These guarantee agronomic feasibility. Examples include the minimum return time for a crop on the same plot, or respecting the biophysical properties of the soil.
- **Soft constraints :** These reflect the farmer's preferences. Examples include the spatio-temporal balance of the crops, or the beneficial effects of previous crops on the soil.

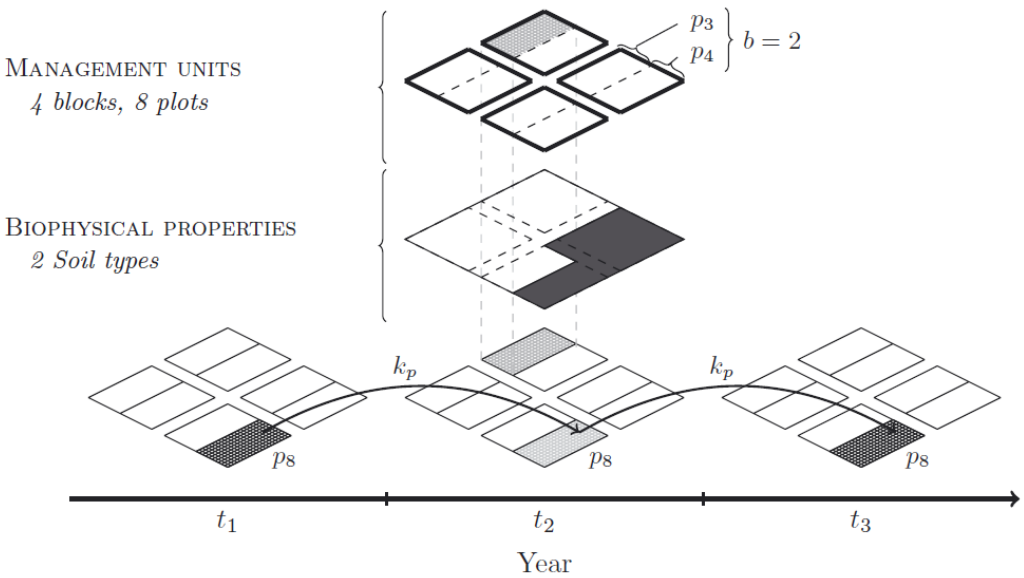

Figure 4: Schematic representation of the spatial and temporal aspects of the decision–making problem (t_i : year, b : block, p_j : plot, k_p : preceding effect)[1].

To solve this problem, the authors compared two approaches. The first is a **WCSP!** formulation solved by Toulbar2. The second is a **MILP!** reformulation solved by SCIP[1].
On a theoretical level, this study highlights a link with local consistency algorithms. The continuous relaxation of the **MILP!** corresponds to the dual problem of the **LP!** (**LP!**) encoded by **OSAC!**. When the upper bound is infinite, this relaxation is theoretically stronger than the **EDAC!** consistency used by default in Toulbar2. However, if this upper bound decreases to a finite value, the two algorithms become incomparable.
However, the experiments reveal a major limitation of WCSP-based methods in this specific context. On real-world instances corresponding to medium and large farms, the SCIP solver proved nearly three times faster than Toulbar2. Furthermore, it consumed significantly

[1]For **MIP!** (**MIP!**) and **MINLP!** (**MINLP!**), SCIP is now among the quickest non-commercial solvers. Additionally, it is a framework for branch-cut-and-price and constraint integer programming.

fewer computational resources.

This study demonstrates a clear conclusion. WCSPs offer excellent flexibility for modeling complex spatio-temporal constraint networks. However, on large-scale, pure scheduling problems, the Toulbar2 solver faces real competition from classic **MILP!** solvers.

In a much more dynamic spatio-temporal context, Schmidt et al. (2020)[3] apply the **WCSP!** formalism to air traffic conflict resolution. A collision risk may be detected between several aircraft. When this happens, a generation method called "Lumberjack" produces a vast set of discretized alternative trajectories for each plane. The combinatorial challenge is then to select the best combination of compatible flight paths. This ensures the conflict is resolved optimally.

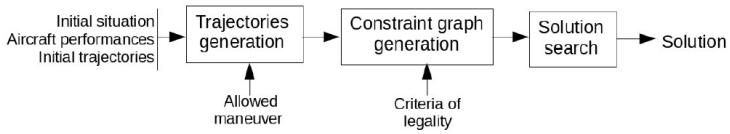

Figure 5: Lumberjack method steps.[3]

In this model, each aircraft represents a variable of the problem, and the set of its alternative avoidance trajectories constitutes its domain of values.

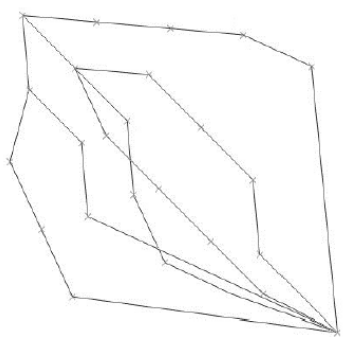

Figure 6: Potential trajectories going from upper left to lower right.[3]

The study compares the performance of the exact solver Toulbar2 with a custom-built "**SBF!** (**SBF!**)" algorithm. Analyzing the results highlights a characteristic trait of the **CFN!** (**CFN!**) architecture and Toulbar2. The solver handles scaling the number of variables perfectly well. It efficiently resolves conflicts involving a relatively high number of aircraft (6 to 8 planes). In contrast, the **SBF!** approach collapses under these conditions.
However, Toulbar2's memory saturates quickly when given too many possible trajectories per plane. In other words, it struggles if the domain size becomes too large. This issue can be explained by two factors. First is the memory cost of representing constraints in extension. Second is the heavy cost of applying local consistency algorithms to very large domains.

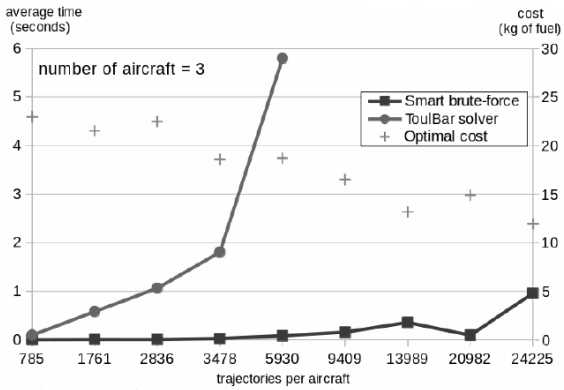

Figure 7: Average solving time over 100 tests of the **SBF!** (squares) and Toulbar2 (circles) approaches for instances of a Roundabout with 3 aircraft, function of the number of generated trajectories per aircraft. Crosses represent the average cost of the optimal solution.[3]

To overcome these limitations, the authors highlight a pruning method using a "greedy" MinCostMaxClique[??] algorithm. Before launching the exact search, they use a fast, non-optimal greedy heuristic to find an initial valid solution. The cost calculated from this solution is then injected into Toulbar2 as an initial "upper bound". This trick allows the solver to prune the search tree much faster. It removes a considerable number of bad solutions and significantly accelerates the proof of optimality.

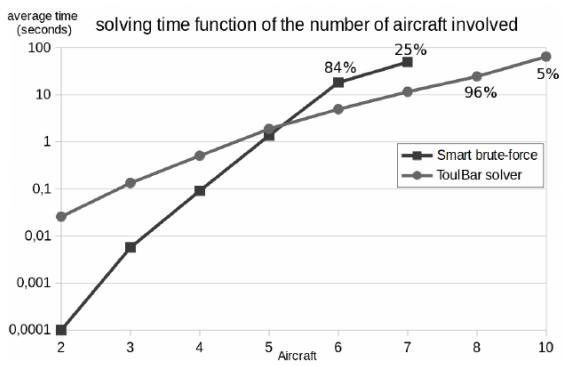

Figure 8: Average solving time over 100 tests with statistical noise, of **SBF!** (squares) and Toulbar2 (circles) approaches for instances of a Roundabout with approx. 850 trajectories per aircraft, function of the number of aircraft involved. When a method does not always succeed before the $60's$ time-out, its success rate is indicated.[3]

This use case shows that Toulbar2's performance depends heavily on the problem's topology. It also highlights the importance of hybridizing exact methods with fast, non-optimal heuristics. This combination guarantees response times that meet real-world industrial requirements.

2.3 Human resources and global constraints

Managing space and time is a major challenge. However, human resources planning introduces a different complexity. It requires modeling legal rules and individual preferences. This subsection highlights how **WCSP!** research has had to evolve. It demonstrates the adaptation required to meet the expressivity needed to generate schedules.

Hospital staff scheduling is an optimization problem known for its high combinatorial complexity. In their study, Métivier et al. (2009)[5] apply the **CFN!** formalism and the Toulbar2

solver to generate optimal schedules for nurses. They must account for strict staff coverage requirements (hard constraints) as well as staff rest preferences (soft constraints).

The value of this document for our analysis lies in demonstrating the historical limitations of **WCSP!**s for this type of problem. Historically, **CFN!**s were designed around binary constraints. However, this type of constraint is poorly suited for modeling human rules. It would require creating an exponentially large constraint network. This makes the solution inefficient.

To overcome this issue, research around toulbar2 focused on implementing soft global constraints. Unlike a binary constraint, a global constraint simultaneously evaluates a large sequence of assignments. For example, it can encompass a nurse's entire week or month in a single evaluation. Métivier et al. (2009) explain how specific algorithms, based on dynamic programming or flow networks, were integrated directly into Toulbar2. This integration allows the application of local consistencies (such as **EDAC!**).

This article demonstrates a significant evolution. The **WCSP!** paradigm is often associated with structural or biological problems. However, it has now achieved a much richer expressivity. Integrating soft global constraints into toulbar2 proves its true capability. It can now successfully compete with traditional **CP!** (**CP!**) solvers in the field of human scheduling.

2.4 Routing and networks

This final section addresses the allocation of purely abstract and logical resources. The networking and telecommunications field provides an excellent testing ground. It allows us to compare **WCSP!**s against other major combinatorial optimization standards.

In their study, Lesaint et al. (2014)[4] examine a complex problem in telecommunications network allocation and configuration : service subscription conflict management. The issue is as follows. A customer may demand a combination of network services that are technically incompatible due to infrastructure limits. When this happens, the provider must find the best compromise. This involves downgrading the service or ignoring certain requested options. The goal is to minimize the overall penalty corresponding to customer dissatisfaction. This NP-hard decision problem was modeled as a **WCSP!**.

The value of this article lies in its comparative analysis. Instead of isolating Toulbar2, the authors systematically compared the **WCSP!** formulation against competing models. These included **MILP!** and Partial Weighted Max-SAT. The results demonstrate the strength of the **WCSP!** approach. By leveraging constraint propagation, it proves highly competitive for dynamically resolving these allocation conflicts.

This operations research use case provides empirical proof. **WCSP!** modeling can compete with historically undisputed paradigms like **MILP!** or Max-SAT for allocating abstract resources. Toulbar2 can sometimes saturate on certain purely linear problems. However, its modeling flexibility and computational power make it an effective alternative for resolving conflicting query satisfaction problems.

2.5 Conclusion

As seen in previous applications, the Toulbar2 solver excels at optimization and planning in deterministic environments. However, resource availability can become uncertain and dynamic. A prime example is IoT sensors, whose energy relies on random environmental conditions (Biason et al., 2017)[2]. In these situations, exact planning shows its limitations. It then becomes necessary to frame the problem using Markov Decision Processes. Here, Toulbar2 is no longer used as a standalone tool. Instead, it serves as an "internal computing engine" powering a probabilistic algorithm.

This need to hybridize exact constraint optimization with models that handle uncertainty marks a natural progression. It directly drives the integration of **WCSP!**s into the broader fields of Artificial Intelligence and Machine Learning.

3 Artificial Intelligence & Machine Learning

4 Conclusion

5 Appendix

A Algorithms and Code Snippets

A.1 MinCostMaxClique algorithm

Algorithm 1: MinCostMaxClique

Where :
– p, the number of aircraft
– Vect, a *structure* representing a tuple of trajectories, and composed of an array *tab* of size p, containing the id of nodes from the p-partite graph ; an *index* ; and a *cost*
– H, a binary min heap of Vect sorted by their *cost*

```
Vect v; v';
v.tab[0..p − 1] := −1; v.index := 0;
v.tab[v.index] + +;
H.insert(v);
while H isn't empty do
    v := H.extract_top;
    if v is legal then
        if v is solution then
            return v;
        v' := v;
        v'.index + +; v'.tab[index] + +;
        H.insert(v');
        v.tab[index] + +;
    H.insert(v);
return "No solution";
```

A.2 Exemple Python Implementation

```python
def iterative_local_search(S0, I_max):
    best_S = S0
    i = 0
    while i < I_max:
        S_prime = perturbate(best_S)
        S_second = local_search(S_prime)

        if cost(S_second) < cost(best_S):
            best_S = S_second

        i += 1
    return best_S
```

Listing 1: Python implementation of the heuristic

References

[1] Mahuna Akplogan et al. "Solving the Crop Allocation Problemusing Hard and Soft Constraints". en. In: *RAIRO - Operations Research* 47.2 (Apr. 2013), pp. 151–172. ISSN: 0399-0559, 1290-3868. DOI: 10.1051/ro/2013032. URL: https://www.cambridge.org/core/journals/rairo-operations-research/article/abs/solving-the-crop-allocation-problemusing-hard-and-soft-constraints/928250B2BF7802F29B31EE47D2FD9993 (visited on 02/04/2026).

[2] Alessandro Biason, Subhrakanti Dey, and Michele Zorzi. "A Decentralized Optimization Framework for Energy Harvesting Devices". In: *IEEE Transactions on Mobile Computing* 17.11 (Nov. 2018), pp. 2483–2496. ISSN: 1558-0660. DOI: 10.1109/TMC.2018.2810269.

[3] Thomas Chaboud, Cédric Pralet, and Nicolas Schmidt. "Tackling Air Traffic Conflicts as a Weighted CSP: Experiments with the Lumberjack Method". In: *arXiv preprint arXiv:2001.11390* (2020).

[4] David Lesaint et al. "Developing Approaches for Solving a Telecommunications Feature Subscription Problem". In: *arXiv e-prints* (2014), arXiv–1401.

[5] Jean-Philippe Métivier, Patrice Boizumault, and Samir Loudni. "Solving Nurse Rostering Problems Using Soft Global Constraints". In: Sept. 2009, pp. 73–87. ISBN: 978-3-642-04243-0. DOI: 10.1007/978-3-642-04244-7_9.